

Homework 3: Poisson Reconstruction

General instructions:

1. Form groups of three students. If not possible, groups of two are allowed. Groups of four are not permitted (please do not ask for exceptions).
2. You are allowed to use AI tools at any time. However, the evaluation of your project will be based not only on the performance of your implementation, but also on an interview of each group with all instructors. It is therefore absolutely essential that you both verify what the AI provides and understand it thoroughly, down to the smallest details.
3. The code will be graded on 20 points according to the structure detailed in this document. This grade is initially attributed to the group. The interview will allow us to objectify this grade and to adjust it individually, depending on each member's performance during the discussion. Adjustments may be made both downwards (in case of insufficient understanding) and upwards (in case of outstanding individual insight).
4. You must submit a short report of at most one page explaining how you organized your work. Be precise about “who did what”. Each member of the group must sign this report.
5. **Important notice.** We will use all anti-plagiarism tools at our disposal. Any confirmed case of plagiarism will result in a grade of zero for both the group who copied and the group whose work was copied.

Reconstructing a continuous and watertight surface from a discrete set of points is a fundamental problem in geometry processing and 3D scanning. The Poisson Surface Reconstruction method provides a global, smooth and robust solution based on solving a Poisson equation that integrates the normal field of an oriented point cloud. This short note presents the main ideas, mathematical formulation, and the preliminary step of estimating normals from unorganized point clouds.

1 Introduction

Surface reconstruction aims to infer a continuous surface $\mathcal{S}$ from a discrete set of samples $\{p_i, n_i\}_{i=1}^N$, where $p_i \in \mathbb{R}^3$ are points lying on the unknown surface and n_i are their corresponding unit normals.

Unlike local methods (e.g. Delaunay-based or moving least squares), Poisson reconstruction formulates the problem globally: it seeks a volumetric function whose gradient best approximates the input normal field. The reconstructed surface is then extracted as an iso-surface of this volumetric function.

2 Normal estimation from a raw point cloud

In most practical scenarios, only the 3D coordinates $\{p_i\}$ are available. The normal vectors $\{n_i\}$ must first be estimated. The standard approach is to compute a local tangent plane around each point.

For each sample p_i , consider its k nearest neighbors $\mathcal{N}_i$. The goal is to approximate the local surface by a plane that best fits these neighboring points in the least-squares sense. The plane can be written as

$$(\mathbf{x} - \mathbf{c}_i) \cdot \mathbf{n}_i = 0,$$

where $\mathbf{c}_i$ is the centroid of the neighborhood,

$$\mathbf{c}_i = \frac{1}{|\mathcal{N}_i|} \sum_{p_j \in \mathcal{N}_i} p_j,$$

and $\mathbf{n}_i$ is the unit normal to the plane.

The optimal normal $\mathbf{n}_i$ is the eigenvector associated with the *smallest eigenvalue* of the covariance matrix

$$\mathbf{C}_i = \sum_{p_j \in \mathcal{N}_i} (p_j - \mathbf{c}_i)(p_j - \mathbf{c}_i)^T.$$

This eigenvector represents the direction of least variation of the local point distribution.

To ensure a globally consistent orientation of normals (all pointing either inward or outward), one may propagate orientations along a graph connecting neighboring points, flipping normals that are inconsistent with already oriented neighbors. A minimum spanning tree traversal is typically used for this propagation.

3 Mathematical formulation of Poisson reconstruction

Let $\Omega \subset \mathbb{R}^3$ denote the interior of the solid whose boundary is the target surface $\mathcal{S} = \partial\Omega$. We introduce the *indicator function* of the solid:

$$\chi(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} \in \Omega, \\ 0 & \text{otherwise.} \end{cases}$$

The gradient of χ is zero almost everywhere, except near the surface, where it aligns with the surface normal:

$$\nabla\chi(\mathbf{x}) \approx \mathbf{n}(\mathbf{x}).$$

We define a continuous vector field $\mathbf{V}(\mathbf{x})$ from the point samples:

$$\mathbf{V}(\mathbf{x}) = \begin{cases} \mathbf{n}_i & \text{near } p_i, \\ \mathbf{0} & \text{elsewhere.} \end{cases}$$

We then seek a scalar function χ whose gradient best matches $\mathbf{V}$ in a least-squares sense:

$$E(\chi) = \int_{\mathbb{R}^3} \|\nabla\chi - \mathbf{V}\|^2 d\mathbf{x}.$$

Minimizing this energy leads to the Poisson equation

$$\Delta\chi = \nabla \cdot \mathbf{V},$$

whose solution χ defines a scalar field. The reconstructed surface corresponds to the iso-surface $\chi = \frac{1}{2}$.

4 Numerical solution

In practice, the domain is discretized on a grid covering the input points. The divergence $\nabla \cdot \mathbf{V}$ is computed at the grid nodes, and the resulting sparse linear system is solved with standard multigrid or conjugate-gradient solvers. The reconstructed surface is then extracted using the Marching Cubes algorithm applied to χ .

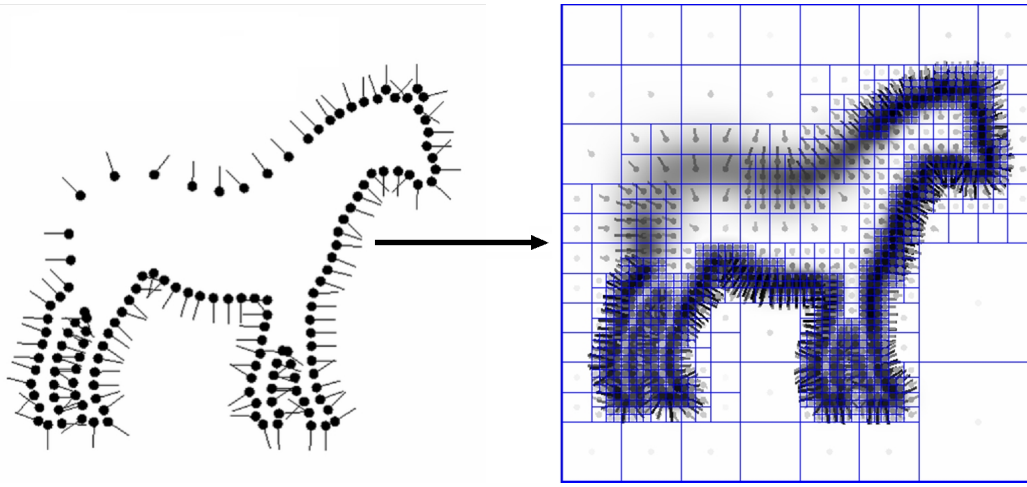


Figure 1: From oriented points to a watertight surface: (A) estimate and orient normals; (B) build a vector field $\mathbf{V}$; (C) solve $\Delta\chi = \nabla \cdot \mathbf{V}$ and extract the iso-surface $\chi = 0.5$.

5 Assignment and Deadline

The goal of this third assignment is to implement a Poisson reconstruction algorithm in dimension 3. The different steps of this assignment are the following:

1. Retrieve a 3D point cloud. There exist many public datasets, for example 3Dscanrep, but numerous alternatives are available. One dataset has been sent to you (see Figure 2).
2. Find a Python package capable of reading your dataset. The formats `ply` and `msh` are very common, and several packages exist to read and write them (e.g. `from plyfile import PlyData`).
3. From the dataset, build a spatial search structure of the kNN-tree type, which will allow you to efficiently find, for each point (`from scipy.spatial import KDTree`), its N closest neighbours. The integer N is a parameter of your program and should be selectable from the command line.
4. Compute at each point an un-oriented normal using the PCA-like method described above.
5. Orient these normals using the method of your choice. You may rely on `libigl` and its fast winding number functionality, or use the BFS procedure described earlier.
6. Solve the Poisson equation on a regular grid using a centred finite-difference scheme (see the lecture slides). The finer the grid, the sharper the resulting iso-surface will be.
7. Use an algorithm of your choice to visualise the resulting iso-surface.
8. Bonus – Use a fast marching algorithm (`pip install scikit-fmm`) to triangulate the surface and save it in `ply` or `msh` format.

The deadline for the assignment is **January 15th, 2026**. You may submit your code by email at jean-francois.remacle@uclouvain.be.

Program Structure

Your program consists mainly of one file, `poisson.py`.

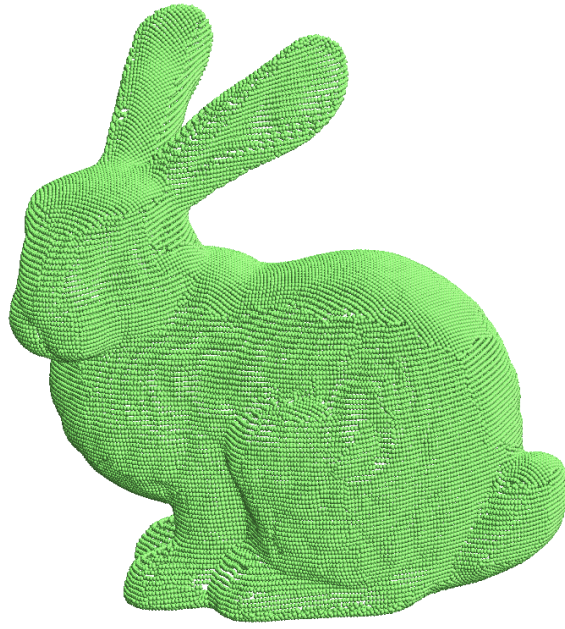


Figure 2: Stanford Bunny

I should be able to **compute** and **visualize** the reconstructed surface for a point cloud `points.ply`. I want to use 10 neighbors to compute the non oriented normals. My uniform grid will be made of 400^3 grid points. Bonus – I want to save the triangulated surface in file `triangles.ply`.

```
python poisson.py -N 10 -M 400 -i points.ply -o triangles.ply
```